

파이썬 임포트 시스템 심층 분석

모듈, 패키지, 그리고 임포트 메커니즘의 이해

발표 내용

1. 임포트 시스템 개요
2. 모듈과 패키지
3. 임포트 검색 과정
4. 모듈 로딩 단계
5. 심화 주제

0. 요약

import 문이 실행되면, Python은 내장 함수 `__import__()` 를 호출하고, 이 함수는 내부적으로 `importlib._bootstrap._find_and_load()` 함수를 사용하여 임포트 과정을 처리합니다. 이 과정에서 먼저 `sys.modules` 딕셔너리에서 모듈을 찾고, 없으면 `sys.meta_path` 의 메타파인더들을 순차적으로 호출하여 모듈을 검색합니다. 파인더가 모듈을 찾으면 `ModuleSpec` 객체를 반환하고, 이 객체를 사용하여 모듈 객체를 생성합니다. 생성된 모듈은 로더의 `exec_module()` 메서드로 초기화되며, 초기화 전에 `sys.modules` 딕셔너리에 {모듈 이름: 모듈 객체} 형태로 추가됩니다.

1. 임포트 시스템 개요

임포트란?

- 한 모듈이 다른 모듈의 코드에 접근하는 과정
- 코드 재사용과 모듈화 프로그래밍의 핵심
- 클린 코드와 유지보수성 향상에 기여

임포트 문의 동작 방식

1. 지정된 모듈 검색
2. 검색 결과를 로컬 스코프의 이름에 바인딩

```
import foo                # 기본 임포트  
from foo import bar       # 특정 속성만 임포트, 근데 과연 애만?  
from foo import bar as baz # 다른 이름으로 임포트
```

임포트 과정의 두 가지 단계

1. 검색(Finding): 모듈 찾기 과정

- 파인더(Finder) 객체 사용

2. 로딩>Loading): 모듈 실행 및 초기화

- 로더(Loader) 객체 사용

고급 임포트 API

`__import__()` 함수

- 내장 함수
- import machinery의 핵심
- 직접 사용은 권장되지 않음

`importlib.import_module()`

- 더 간단하고 권장되는 API
- 내부적으로 `__import__()` 호출

권장되는 방법

```
import importlib  
module = importlib.import_module('foo.bar')
```

2. 모듈과 패키지

모듈(Module)의 개념

- 파이썬 코드를 구성하는 기본 단위
- 파이썬 파일(.py)이 모듈이 될 수 있음
- 코드 조직화와 네임스페이스 관리
- `import` 문으로 가져와 사용

패키지(Package)의 개념

- 모듈을 담는 컨테이너
- 계층적 구조로 모듈 조직화
- "디렉토리 = 패키지, 파일 = 모듈" 구조
- **중요:** 모든 패키지는 모듈이지만, 모든 모듈이 패키지는 아님

정규 패키지 (Regular Package)

- `__init__.py` 파일이 포함된 디렉토리
- 전통적인 패키지 구조 (Python 3.2 이전 방식)
- 패키지 초기화 코드 실행
- 패키지 수준의 변수와 함수 정의 가능

```
parent/  
  __init__.py  
  one/  
    __init__.py  
  two/  
    __init__.py
```

네임스페이스 패키지 (Namespace Package)

- `__init__.py` 파일이 없음
- 여러 경로에 걸쳐 존재 가능
- PEP 420에서 도입 (Python 3.3부터)
- 초기화 코드 없음

```
path1/  
    parent/  
        one/  
            module1.py  
path2/  
    parent/  
        two/  
            module2.py
```

정규 패키지 vs 네임스페이스 패키지

특성	정규 패키지	네임스페이스 패키지
<code>__init__.py</code>	필수	없음
위치	단일 디렉토리	여러 디렉토리 가능
초기화 코드	있음	없음
<code>__file__</code> 속성	있음	없음
<code>__path__</code> 속성	단일 경로	여러 경로(<code>_NamespacePath</code>)

네임스페이스 패키지 예시 (프로젝트 구조)

```
my-library-core/  
  mylib/  
    core/  
      __init__.py  
      utils.py
```

```
my-library-extras/  
  mylib/  
    extras/  
      __init__.py  
      feature.py
```

네임스페이스 패키지 예시 (사용)

```
# 두 패키지가 설치되었을 때
from mylib.core import utils
from mylib.extras import feature

# 'mylib'은 네임스페이스 패키지
```

3. 임포트 검색 과정

모듈 검색 순서

1. `sys.modules` 캐시 확인
2. 내장 모듈 확인
3. `sys.meta_path` 의 파인더들에게 검색 요청
4. 못 찾으면 `ModuleNotFoundError` 발생

모듈 캐시 (`sys.modules`)

- 이미 임포트된 모듈을 저장하는 딕셔너리
- 임포트 과정에서 가장 먼저 확인
- 키: 모듈 이름, 값: 모듈 객체



```
>> sys.modules
```

```
{'sys': <module 'sys' (built-in)>,  
'builtins': <module 'builtins' (built-in)>,  
'_frozen_importlib': <module '_frozen_importlib'  
(frozen)> module '_imp' (built-in)>,  
'_thread': <module '_thread' (built-in)>, ...}
```

파인더(Finder)와 로더(Loader)

- 파인더: 모듈을 찾는 객체
 - `find_spec()` 메서드 구현
 - 모듈 스펙(ModuleSpec) 반환
- 로더: 모듈을 로드하는 객체
 - `create_module()`, `exec_module()` 메서드 구현
 - 모듈 초기화 담당

메타 패스 (`sys.meta_path`)

- 메타 패스 파인더 목록
- 등록된 순서대로 `find_spec()` 호출
- 기본 메타 파인더:
 - i. 내장 모듈 파인더 (Built-in)
 - ii. 프로즌 모듈 파인더 (Frozen)
 - iii. 경로 기반 파인더 (Path-based)



```
>> sys.metapath
```

```
[<class '_frozen_importlib.BuiltinImporter'>,  
<class '_frozen_importlib.FrozenImporter'>,  
<class  
'_frozen_importlib_external.PathFinder'>]
```

경로 기반 파인더 (PathFinder)

- `sys.path` 와 `sys.path_hooks` 사용
- 파일 시스템, zip 파일, URL 등에서 모듈 검색
- 서브 파인더 (path entry finder)들에게 실제 검색 위임

4. 모듈 로딩 단계

모듈 스펙 (ModuleSpec)

- 모듈의 로딩 관련 정보 캡슐화
- 핵심 속성:
 - `name` : 모듈 이름
 - `loader` : 로더 객체
 - `origin` : 모듈 소스 위치
 - `submodule_search_locations` : 서브모듈 검색 경로

모듈 로딩 의사코드

```
# 모듈 객체 생성
if spec.loader is not None and hasattr(spec.loader, 'create_module'):
    module = spec.loader.create_module(spec)
if module is None:
    module = ModuleType(spec.name)

# 모듈 속성 초기화
_init_module_attrs(spec, module)

# 모듈 등록 및 실행
sys.modules[spec.name] = module
try:
    spec.loader.exec_module(module)
except BaseException:
    del sys.modules[spec.name]
    raise
```

모듈 로딩 과정의 주요 단계

1. 모듈 객체 생성

- 로더의 `create_module()` 또는 기본 `ModuleType`

2. 모듈 속성 초기화

- `__name__`, `__loader__`, `__package__`, `__spec__` 등

3. 모듈 코드 실행

- 로더의 `exec_module()` 메서드 호출
- 네임스페이스 패키지는 실행 없음

중요한 로딩 세부사항

- 모듈은 `sys.modules` 에 먼저 등록됨 (순환 참조 방지)
- 로딩 실패 시 해당 모듈만 `sys.modules` 에서 제거
- 성공적으로 로드된 부수적 모듈은 캐시에 유지
- Python 3.4부터 로더의 책임 대부분이 interpreter machinery로 이동

5. 심화 주제

namespace 패키지

내가 zope.interface 패키지 안에서 사용자 정의 무언가를 만들고 싶어!

```
zope-interface-proxy/  
  setup.py  
  zope/                # 네임스페이스 패키지 (init.py 없음)  
    interface/        # 네임스페이스 패키지 (init.py 없음)  
      proxy/          # 실제 패키지  
        __init__.py  
        proxy.py
```

setup.py 설정

```
from setuptools import setup, find_namespace_packages

setup(
    name="zope-interface-proxy",
    version="0.1.0",
    packages=find_namespace_packages(include=["zope.*"]),
    namespace_packages=["zope", "zope.interface"],
)
```

네임스페이스 패키지를 통해 접근하기

```
import zope.interface.proxy  
# 또는  
from zope.interface.proxy import SomeClass
```

패키지 상대 импорт

- 선행 점(.)을 사용한 импорт
- 한 개의 점: 현재 패키지 기준
- 두 개 이상의 점: 상위 패키지 기준

```
# subpackage1/moduleX.py 내에서
from .moduleY import spam          # 같은 패키지
from ..subpackage2 import moduleZ  # 부모의 다른 서브패키지
from ... import moduleA            # 최상위 패키지의 모듈
```

`__main__` 모듈의 특별한 처리

- 인터프리터 시작 시 직접 초기화됨
- 실행 방식에 따라 설정이 달라짐:
 - `-m` 옵션: 해당 모듈의 스펙 설정
 - 직접 실행: `__spec__` 는 `None`
- `if __name__ == "__main__":` 블록은 직접 실행 시에만 작동

모듈 던더(Dunder) 속성

속성	설명
<code>__name__</code>	모듈의 이름
<code>__file__</code>	모듈 파일의 경로
<code>__package__</code>	모듈이 속한 패키지 이름
<code>__path__</code>	패키지의 서브모듈 검색 경로
<code>__spec__</code>	모듈 스펙 정보
<code>__loader__</code>	모듈을 로드한 로더 객체

정리: 임포트 시스템 핵심 개념

1. 임포트는 검색과 로딩 두 단계로
2. 모듈 캐시 (`sys.modules`)가 첫 번째 검색 대상
3. 메타 패스 파인더들이 차례로 모듈을 검색
4. 모듈 스펙이 로더에게 전달되어 모듈 로드
5. 네임스페이스 패키지와 정규 패키지는 다르게 처리

참고 자료

- 파이썬 공식 문서: импорт 시스템
- PEP 302: импорт 후크
- PEP 420: 네임스페이스 패키지
- PEP 451: 모듈 스펙 객체 도입

질의응답

발표 내용에 대한 질문이 있으시면 말씀해 주세요.